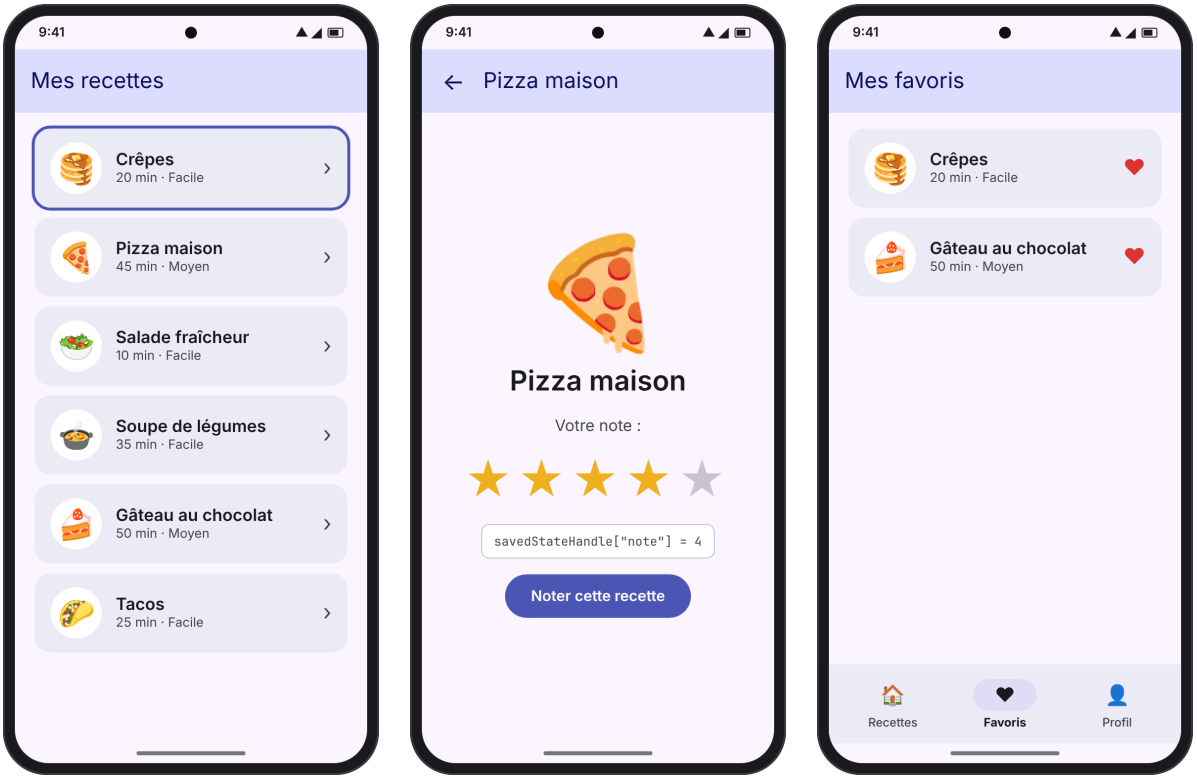
COURS DÉBUTANT · ANDROID

# La navigation avec Jetpack Compose

Passer d'un écran à l'autre, transmettre des informations, gérer le bouton Retour et créer une barre d'onglets — pas à pas, avec des exemples simples et des captures d'écran.

Kotlin | Jetpack Compose | Navigation Compose | Projet Android Studio inclus



# Sommaire


| | | |
|---|---|---|
| 1 | Les idées de base | vocabulaire |
| 2 | Préparer le projet | Gradle |
| 3 | Exemple 1 — Aller d'un écran à l'autre | navigate |
| 4 | Exemple 2 — Passer une information | arguments |
| 5 | Exemple 3 — Maîtriser la pile d'écrans | popUpTo |
| 6 | Exemple 4 — Une barre d'onglets en bas | NavigationBar |
| 7 | Exemple 5 — Renvoyer un résultat | SavedStateHandle |
| 8 | Les bonnes pratiques | et pour aller plus loin |
| 9 | Les erreurs fréquentes du débutant | et leurs solutions |
| 10 | Exercices corrigés | pratique |
| 11 | Aide-mémoire | à garder sous la main |
| 12 | Le projet Android Studio | mode d'emploi |


**À qui s'adresse ce cours ?**

À toute personne qui connaît les bases de Kotlin et qui sait afficher un texte et un bouton avec Compose ( `@Composable` , `Text` , `Button` , `Column` ). Aucune connaissance de la navigation n'est nécessaire.

**Un projet prêt à lancer**

Tout le code de ce cours se trouve dans le projet Android Studio **MesRecettes** fourni avec ce PDF (dossier `projet/` ). Chaque exemple s'ouvre depuis un menu. Le chapitre 12 explique comment l'ouvrir.

**À propos des captures d'écran**

Les captures sont des rendus fidèles des écrans produits par le code de chaque exemple (thème Material 3 par défaut). Selon votre téléphone et votre version d'Android, l'apparence peut varier légèrement.

CHAPITRE 1

# Les idées de base

Presque toutes les applications ont plusieurs écrans : une liste, un détail, un profil, des paramètres… **Naviguer**, c'est passer de l'un à l'autre — et savoir revenir en arrière.

Avec Jetpack Compose, un écran n'est rien d'autre qu'une **fonction** `@Composable`. La bibliothèque **Navigation Compose** s'occupe de choisir quelle fonction afficher. Elle repose sur quatre notions :

| Notion | Image | Rôle |
|---|---|---|
| **Destination** (ou route) | Une adresse sur une carte | Le nom d'un écran. En Kotlin, c'est un `object` ou une `data class` marqué `@Serializable`. |
| **NavHost** | La carte de la ville | Le Composable qui liste **toutes** les destinations et affiche celle qui est active. |
| **NavController** | Le chauffeur | L'objet qui **change** d'écran : `navigate(...)` pour avancer, `popBackStack()` pour reculer. |
| **Pile de retour** (back stack) | Une pile d'assiettes | La liste des écrans visités. Le nouvel écran est posé **dessus** ; « Retour » retire celui du dessus. |

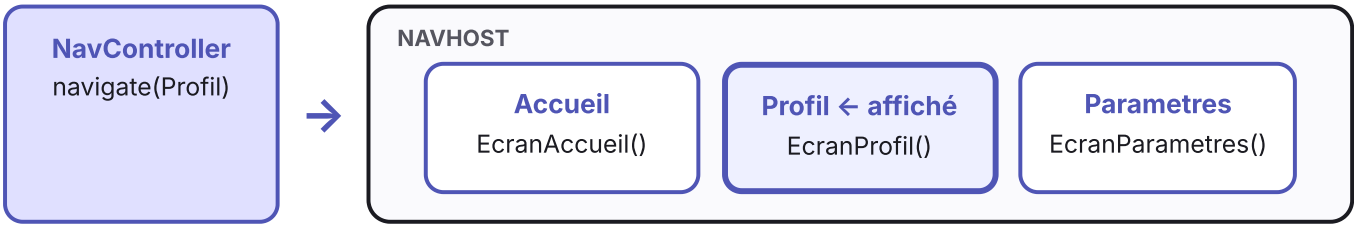
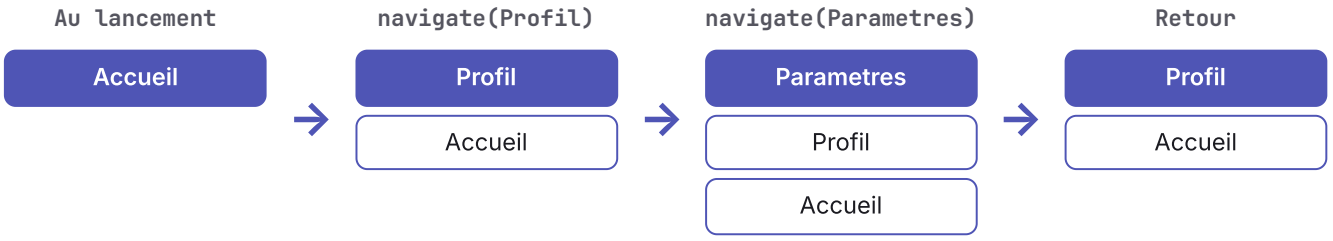



Schéma : le NavController demande au NavHost d'afficher une destination.

## La pile de retour, en images



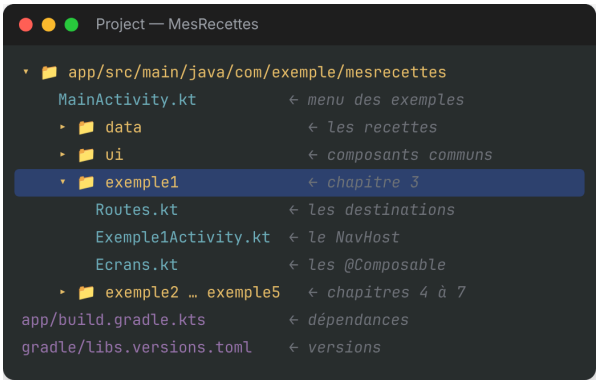

L'écran affiché est toujours celui **du haut** de la pile (en couleur). Quand la pile ne contient plus qu'un écran, « Retour » ferme l'application.

> **Navigation « typée »**
>
> Depuis la version 2.8, Navigation Compose utilise des **classes Kotlin** comme destinations, au lieu de chaînes de caractères comme `"detail/{id}"`. Le compilateur vérifie alors vos erreurs à votre place. C'est la méthode recommandée, celle de ce cours. Sur Internet, vous verrez encore l'ancienne syntaxe avec des chaînes : le principe est le même.

CHAPITRE 2

# Préparer le projet

Partez d'un projet Android Studio créé avec le modèle **Empty Activity** (Compose déjà activé). Il faut ajouter deux choses : la bibliothèque **Navigation Compose** et le plugin de **sérialisation** Kotlin, qui transforme vos destinations en « adresses ».




Capture 1 — Organisation du projet MesRecettes fourni avec le cours.

## Étape 1 — Les versions

gradle/libs.versions.toml (extrait)

```
[versions]
kotlin = "2.0.21"
navigation = "2.8.5"
serialization = "1.7.3"

[libraries]
androidx-navigation-compose = { group = "androidx.navigation", name = "navigation-compose",
version.ref = "navigation" }
kotlinx-serialization-json = { group = "org.jetbrains.kotlinx", name = "kotlinx-serialization-
json", version.ref = "serialization" }

[plugins]
kotlin-serialization = { id = "org.jetbrains.kotlin.plugin.serialization", version.ref =
"kotlin" }
```

## Étape 2 — Le module app

app/build.gradle.kts (extrait)

```
plugins {
    // ... plugins déjà présents
    alias(libs.plugins.kotlin.serialization)  // indispensable pour les routes @Serializable
}

dependencies {
    // ... dépendances déjà présentes
    implementation(libs.androidx.navigation.compose)
```

```
    implementation(libs.kotlinx.serialization.json)
}
```

Dans le fichier `build.gradle.kts` **racine**, déclarez aussi le plugin : `alias(libs.plugins.kotlin.serialization) apply false`. Cliquez enfin sur **Sync Now**.

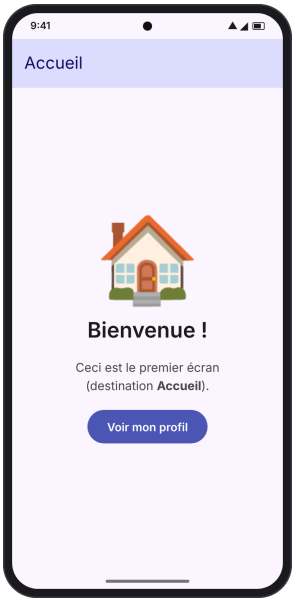
> **Même version que Kotlin**
> Le plugin de sérialisation doit avoir **exactement** la même version que votre plugin Kotlin (ici `2.0.21`). C'est pour cela qu'il réutilise `version.ref = "kotlin"`.

> **Mettre à jour**
> Des versions plus récentes existent sûrement quand vous lisez ce cours. Android Studio souligne les dépendances obsolètes : **Alt + Entrée** propose la mise à jour.

CHAPITRE 3 · EXEMPLE 1

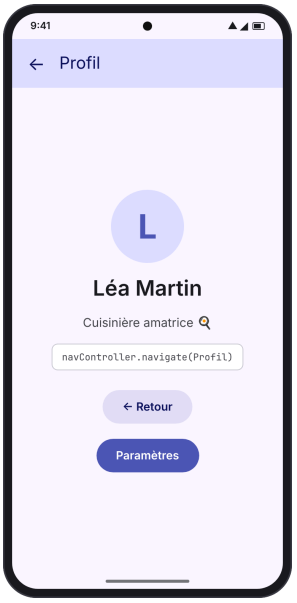
# Aller d'un écran à l'autre

**Objectif :** un écran **Accueil** avec un bouton « Voir mon profil » qui ouvre un écran **Profil**. Le bouton Retour revient à l'accueil.




Capture 2 — Destination Accueil

navigate( Profil)




Capture 3 — Destination Profil

## Étape 1 — Déclarer les destinations

Une destination sans information à transmettre est un simple `object` :

```
exemple1/Routes.kt

// Chapitre 3 : une destination = un objet marqué @Serializable

@Serializable
object Accueil

@Serializable
object Profil

@Serializable
object Parametres      // exercice 1
```

## Étape 2 — Écrire les écrans

Les écrans sont des Composables ordinaires. Point essentiel : ils **ne connaissent pas** le NavController. Ils reçoivent des fonctions ( `onVoirProfil` , `onRetour` ) qu'ils appellent quand on clique.

```
exemple1/Ecrans.kt (extrait)

@Composable
fun EcranAccueil(onVoirProfil: () -> Unit) {
    Scaffold(topBar = { BarreDuHaut("Accueil") }) { padding ->
```

```
        Column(
            Modifier.padding(padding).fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.spacedBy(16.dp, Alignment.CenterVertically)
        ) {
            Text("🏠", fontSize = 96.sp)
            Text("Bienvenue !", style = MaterialTheme.typography.headlineMedium)
            Text("Ceci est le premier écran\n(destination Accueil).", textAlign =
TextAlign.Center)
            Button(onClick = onVoirProfil) { Text("Voir mon profil") }
        }
    }
}

@Composable
fun EcranProfil(onRetour: () -> Unit, onParametres: () -> Unit) {
    Scaffold(topBar = { BarreDuHaut("Profil", onRetour = onRetour) }) { padding ->
        Column(
            Modifier.padding(padding).fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.spacedBy(16.dp, Alignment.CenterVertically)
        ) {
            Text("👨‍🍳", fontSize = 80.sp)
            Text("Léa Martin", style = MaterialTheme.typography.headlineMedium)
            Text("Cuisinière amatrice 🍳")
            FilledTonalButton(onClick = onRetour) { Text("← Retour") }
            Button(onClick = onParametres) { Text("Paramètres") }
        }
    }
}
```

Le petit composant `BarreDuHaut(titre, onRetour)` est une `TopAppBar` avec une flèche ← quand `onRetour` est fourni. Il se trouve dans `ui/Composants.kt`.

## Étape 3 — Le NavHost

C'est ici que tout se relie : on crée le NavController, on liste les destinations, et on dit quoi faire quand un écran appelle ses fonctions.

exemple1/Exemple1Activity.kt

```
class Exemple1Activity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { MaterialTheme { AppExemple1() } }
    }
}

@Composable
fun AppExemple1() {
    // 1. Le NavController : le "chauffeur" qui change d'écran
    val navController = rememberNavController()

    // 2. Le NavHost : la "carte" de tous les écrans
    NavHost(navController = navController, startDestination = Accueil) {
```

```
        composable<Accueil> {
            EcranAccueil(
                onVoirProfil = { navController.navigate(Profil) }
            )
        }

        composable<Profil> {
            EcranProfil(
                onRetour = { navController.popBackStack() },
                onParametres = { navController.navigate(Parametres) }   // exercice 1
            )
        }

        composable<Parametres> {                                        // exercice 1
            EcranParametres(onRetour = { navController.popBackStack() })
        }
    }
}
```

## Ce qu'il faut retenir

1. `rememberNavController()` crée le « chauffeur ».
2. `NavHost(navController, startDestination = Accueil) { ... }` liste les écrans ; `startDestination` est le premier affiché.
3. `composable<Accueil> { ... }` associe une destination à un Composable.
4. `navController.navigate(Profil)` avance ; `navController.popBackStack()` recule.

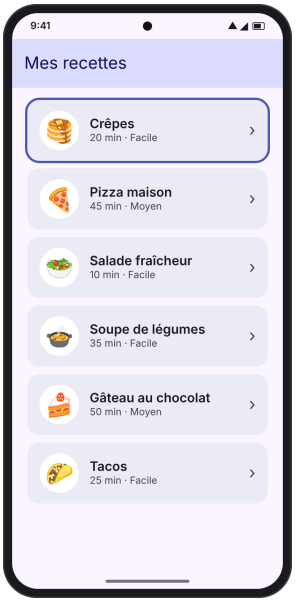
| Méthode | Effet |
|---|---|
| `navigate(destination)` | Pose un nouvel écran sur la pile. |
| `popBackStack()` | Retire l'écran du dessus (comme le bouton Retour du téléphone). |
| `navigateUp()` | Remonte d'un niveau ; presque identique à `popBackStack()` dans une appli simple. Idéal pour la flèche ← de la barre du haut. |

> **Le bouton Retour du téléphone**
>
> Vous n'avez **rien** à coder : le NavHost gère automatiquement le bouton (ou le geste) Retour d'Android.

CHAPITRE 4 · EXEMPLE 2

# Passer une information à un écran

**Objectif :** une liste de recettes. Quand on touche une recette, l'écran de détail doit savoir **laquelle** afficher. On va lui transmettre son identifiant ( `id` ).




Capture 4 — ListeRecettes

navigate(
Detail(id = 1))




Capture 5 — Detail

## Étape 1 — Une destination avec argument = une data class

exemple2/Routes.kt

```
// Chapitre 4 : une destination AVEC argument = une data class

@Serializable
object ListeRecettes

@Serializable
data class Detail(val id: Int)

@Serializable
data class Auteur(val nom: String)      // exercice 2
```

## Étape 2 — Envoyer et recevoir l'argument

exemple2/Exemple2Activity.kt (extrait)

```
@Composable
fun AppExemple2() {
    val navController = rememberNavController()

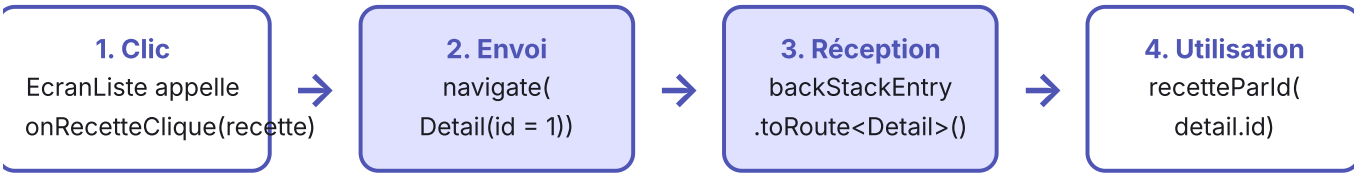
    NavHost(navController = navController, startDestination = ListeRecettes) {

        composable<ListeRecettes> {
            EcranListe(
```

```
                onRecetteClique = { recette ->
                    // On crée la destination AVEC sa valeur
                    navController.navigate(Detail(id = recette.id))
                }
            )
        }

        composable<Detail> { backStackEntry ->
            // On relit l'argument, déjà avec le bon type (Int)
            val detail: Detail = backStackEntry.toRoute()
            val recette = recetteParId(detail.id)
            EcranDetail(
                recette = recette,
                onRetour = { navController.popBackStack() },
                onVoirAuteur = { navController.navigate(Auteur(nom = recette.auteur)) }  //
exercice 2
            )
        }

        composable<Auteur> { backStackEntry ->                                          //
exercice 2
            val auteur: Auteur = backStackEntry.toRoute()
            EcranAuteur(nom = auteur.nom, onRetour = { navController.popBackStack() })
        }
    }
}
```




## Étape 3 — L'écran de détail

L'écran reçoit directement la `Recette` à afficher. Il ne sait pas d'où elle vient : c'est le NavHost qui l'a retrouvée.

exemple2/Ecrans.kt (extrait)

```
@Composable
fun EcranDetail(recette: Recette, onRetour: () -> Unit, onVoirAuteur: () -> Unit) {
    Scaffold(topBar = { BarreDuHaut(recette.nom, onRetour = onRetour) }) { padding ->
        Column(
            Modifier.padding(padding).fillMaxSize().padding(24.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.spacedBy(16.dp, Alignment.CenterVertically)
        ) {
            Text(recette.emoji, fontSize = 96.sp)
            Text(recette.nom, style = MaterialTheme.typography.headlineMedium)
            Text("⏱ ${recette.duree} min · ${recette.difficulte}")
            Text(recette.description, textAlign = TextAlign.Center)
            FilledTonalButton(onClick = onVoirAuteur) { Text("Voir l'auteur") }
        }
```

```
    }
}
```

**Ne transmettez pas un objet entier**

On envoie `Detail(id = 1)`, **pas** la recette complète. Les arguments sont enregistrés par Android (pour survivre à une rotation, par exemple) : ils doivent rester **petits et simples**. On transmet un identifiant, puis on retrouve l'objet (ici avec `recetteParId`, plus tard dans une base de données).

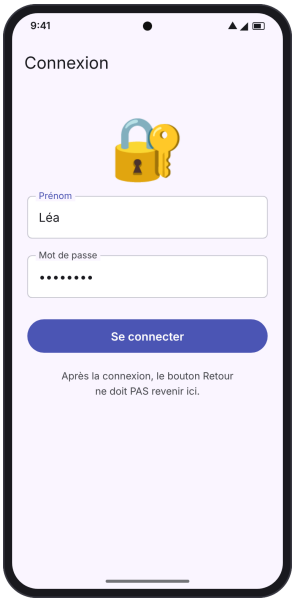
## Quels types peut-on utiliser ?

| Type | Exemple |
|---|---|
| `Int`, `Long`, `Float`, `Double`, `Boolean` | `data class Detail(val id: Int)` |
| `String` | `data class Auteur(val nom: String)` |
| Valeur facultative (avec une valeur par défaut) | `data class Recherche(val texte: String = "")` → `navigate(Recherche())` ou `navigate(Recherche("tarte"))` |
| Valeur qui peut être absente | `data class Filtre(val categorie: String? = null)` |
| Listes simples | `data class Selection(val ids: List<Int>)` |

CHAPITRE 5 · EXEMPLE 3

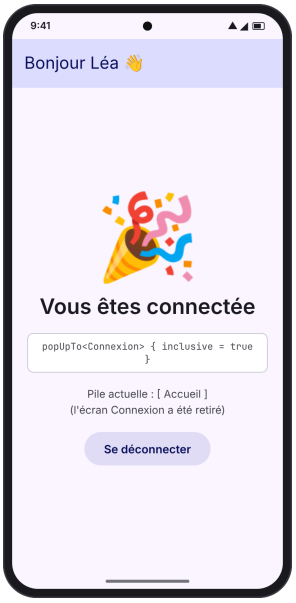
# Maîtriser la pile d'écrans

**Situation :** un écran de **Connexion**. Une fois connecté, on arrive sur l'accueil. Problème : avec un simple `navigate`, un appui sur Retour… ramène à l'écran de connexion ! Ce n'est pas ce qu'on veut.



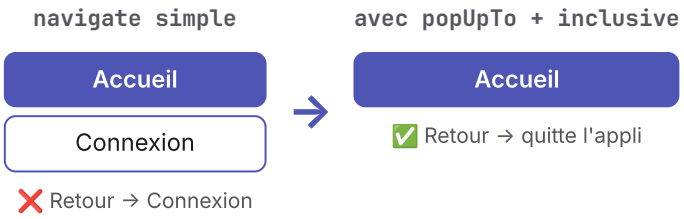

Capture 6 — Connexion

navigate + popUpTo




Capture 7 — Accueil connecté

## Le problème, puis la solution




L'option `popUpTo` retire des écrans de la pile **avant** d'ajouter le nouveau. On l'écrit dans un bloc `{ }` juste après `navigate` :

exemple3/Exemple3Activity.kt (extrait)

```
composable<Connexion> {
    EcranConnexion(
        onConnecte = { prenom ->
            navController.navigate(AccueilConnecte(prenom)) {
                // Retire Connexion de la pile (inclusive = true : Connexion compris)
                popUpTo<Connexion> { inclusive = true }
                // N'empile pas deux fois le même écran en cas de double clic
                launchSingleTop = true
            }
        }
    )
}
```

| Option | Ce qu'elle fait |
|---|---|
| `popUpTo<X>()` | Retire tous les écrans posés **au-dessus** de X. |
| `popUpTo<X> { inclusive = true }` | Pareil, et retire **X lui-même**. |
| `popUpTo(navController.graph.id) { inclusive = true }` | Vide **toute** la pile. |
| `launchSingleTop = true` | Si l'écran demandé est déjà en haut de la pile, on ne le rajoute pas une 2e fois (utile contre les doubles clics). |

## L'écran de connexion

Il garde ce que l'on tape avec `rememberSaveable` (le texte survit à la rotation), puis appelle `onConnecte(prenom)`.

exemple3/Ecrans.kt (extrait)

```
@Composable
fun EcranConnexion(onConnecte: (String) -> Unit) {
    var prenom by rememberSaveable { mutableStateOf("") }
    var motDePasse by rememberSaveable { mutableStateOf("") }

    Scaffold(topBar = { BarreDuHaut("Connexion") }) { padding ->
        Column(
            Modifier.padding(padding).fillMaxSize().padding(24.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.spacedBy(16.dp, Alignment.CenterVertically)
        ) {
            Text("🔐", fontSize = 72.sp)
            OutlinedTextField(
                value = prenom,
                onValueChange = { prenom = it },
                label = { Text("Prénom") },
                singleLine = true,
                modifier = Modifier.fillMaxWidth()
            )
            OutlinedTextField(
                value = motDePasse,
                onValueChange = { motDePasse = it },
                label = { Text("Mot de passe") },
                singleLine = true,
                visualTransformation = PasswordVisualTransformation(),
                keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Password),
                modifier = Modifier.fillMaxWidth()
            )
            Button(
                onClick = { onConnecte(prenom.ifBlank { "invité" }) },
                modifier = Modifier.fillMaxWidth()
            ) { Text("Se connecter") }
            // … petit texte d'aide
        }
```

```
    }
}
```

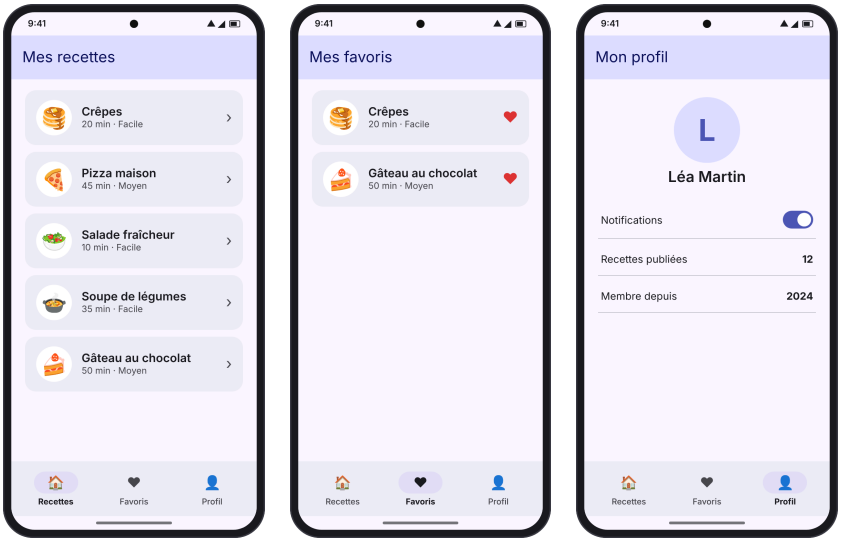
**Et la déconnexion ?**

C'est l'exercice 3 (chapitre 10) : il faut revenir à la connexion en vidant **toute** la pile, sinon Retour ramènerait à l'accueil connecté.

CHAPITRE 6 · EXEMPLE 4

# Une barre d'onglets en bas

**Objectif :** trois onglets — Recettes, Favoris, Profil — toujours visibles en bas de l'écran, comme dans la plupart des applications.




Capture 8 — Recettes  Capture 9 — Favoris  Capture 10 — Profil

## Étape 1 — Les destinations et la liste des onglets

exemple4/Routes.kt

```
// Chapitre 6 : une destination par onglet

@Serializable
object Recettes

@Serializable
object Favoris

@Serializable
object Profil

/** Décrit un onglet de la barre du bas. */
data class Onglet<T : Any>(val titre: String, val icone: ImageVector, val route: T)

val onglets = listOf(
    Onglet("Recettes", Icons.Filled.Home, Recettes),
    Onglet("Favoris", Icons.Filled.Favorite, Favoris),
    Onglet("Profil", Icons.Filled.Person, Profil),
)
```

## Étape 2 — Le Scaffold commun et le NavHost

Cette fois, la barre du haut et la barre du bas sont dessinées **une seule fois**, autour du NavHost. Seul le contenu du milieu change.

exemple4/Exemple4Activity.kt (extrait)

```kotlin
@Composable
fun AppExemple4() {
    val navController = rememberNavController()

    // L'écran actuellement affiché (se met à jour à chaque navigation)
    val entreeActuelle by navController.currentBackStackEntryAsState()
    val destinationActuelle = entreeActuelle?.destination

    // L'onglet sélectionné = celui dont la route correspond à la destination actuelle
    val ongletActuel = onglets.firstOrNull { onglet ->
        destinationActuelle?.hierarchy?.any { it.hasRoute(onglet.route::class) } == true
    }

    Scaffold(
        topBar = { BarreDuHaut(ongletActuel?.titre ?: "") },
        bottomBar = {
            NavigationBar {
                onglets.forEach { onglet ->
                    NavigationBarItem(
                        selected = onglet == ongletActuel,
                        onClick = {
                            navController.navigate(onglet.route) {
                                // Revient au 1er onglet en gardant l'état des autres
                                popUpTo(navController.graph.findStartDestination().id) {
                                    saveState = true
                                }
                                launchSingleTop = true   // pas de doublon si on re-touche
l'onglet
                                restoreState = true      // retrouve l'état (ex. position de
la liste)
                            }
                        },
                        icon = { Icon(onglet.icone, contentDescription = onglet.titre) },
                        label = { Text(onglet.titre) }
                    )
                }
            }
        }
    ) { padding ->
        // Le NavHost occupe l'espace entre les deux barres
        NavHost(
            navController = navController,
            startDestination = Recettes,
            modifier = Modifier.padding(padding)
        ) {
            composable<Recettes> { EcranRecettes() }
            composable<Favoris> { EcranFavoris() }
            composable<Profil> { EcranProfil() }
        }
    }
}
```

## Les trois lignes magiques

| Ligne | Pourquoi ? |
| --- | --- |
| `popUpTo(...findStartDestination().id) { saveState = true }` | Évite d'empiler les onglets à l'infini (Recettes → Favoris → Profil → Favoris…). Retour depuis un onglet ramène au premier, puis quitte l'appli. |
| `launchSingleTop = true` | Toucher l'onglet déjà sélectionné ne crée pas de doublon. |
| `restoreState = true` | En revenant sur un onglet, on retrouve son état : position dans la liste, interrupteur du profil… |

**Savoir quel onglet est actif**

`navController.currentBackStackEntryAsState()` donne l'écran affiché et se met à jour à chaque navigation. On compare sa destination avec la route de chaque onglet grâce à `hasRoute(onglet.route::class)`.

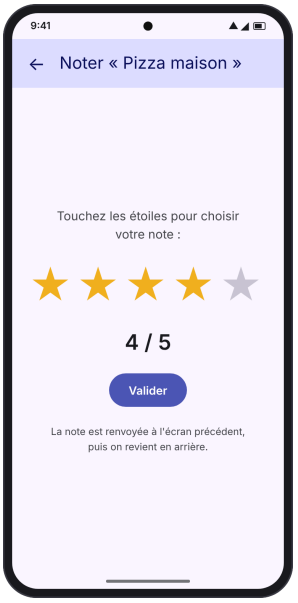
**Pas de Scaffold dans les écrans d'onglets**

Les écrans `EcranRecettes`, `EcranFavoris` et `EcranProfil` n'ont pas leur propre `Scaffold` : sinon on aurait deux barres du haut. N'oubliez pas `Modifier.padding(padding)` sur le NavHost, sinon le contenu passe sous les barres.

CHAPITRE 7 · EXEMPLE 5

# Renvoyer un résultat

**Objectif :** depuis le détail d'une recette, on ouvre un écran **Noter**. L'utilisateur choisit une note de 1 à 5, valide… et la note doit apparaître sur l'écran de détail.




Capture 11 — Noter

note renvoyée
+ popBackStack




Capture 12 — Retour au détail

## Le principe : une « boîte aux lettres »

Chaque écran de la pile possède une petite boîte de rangement, le `savedStateHandle`. L'écran Noter dépose la note dans la boîte de l'écran **précédent**, puis se ferme. L'écran de détail, qui surveille sa boîte, voit la note arriver.




exemple5/Exemple5Activity.kt (extrait)

```
        composable<Detail> { backStackEntry ->
            val recette = recetteParId(backStackEntry.toRoute<Detail>().id)

            // 2. On ÉCOUTE la note déposée dans NOTRE SavedStateHandle (0 = pas encore notée)
            val note by backStackEntry.savedStateHandle
                .getStateFlow(CLE_NOTE, 0)
                .collectAsState()

            EcranDetailAvecNote(
                recette = recette,
                note = note,
                onNoter = { navController.navigate(Noter(recette.id)) },
                onRetour = { navController.popBackStack() }
            )
```

```
        }

        composable<Noter> { backStackEntry ->
            val recette = recetteParId(backStackEntry.toRoute<Noter>().id)
            EcranNoter(
                recette = recette,
                onValider = { noteChoisie ->
                    // 1. On DÉPOSE la note dans le SavedStateHandle de l'écran PRÉCÉDENT…
                    navController.previousBackStackEntry
                        ?.savedStateHandle
                        ?.set(CLE_NOTE, noteChoisie)
                    // … puis on revient en arrière
                    navController.popBackStack()
                },
                onRetour = { navController.popBackStack() }
            )
        }
```

## L'écran Noter

exemple5/Ecrans.kt (extrait)

```
@Composable
fun EcranNoter(recette: Recette, onValider: (Int) -> Unit, onRetour: () -> Unit) {
    var note by rememberSaveable { mutableIntStateOf(3) }
    Scaffold(topBar = { BarreDuHaut("Noter « ${recette.nom} »", onRetour = onRetour) }) {
padding ->
        Column(
            Modifier.padding(padding).fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.spacedBy(24.dp, Alignment.CenterVertically)
        ) {
            Text("Touchez les étoiles pour choisir votre note :")
            Etoiles(note = note, taille = 52.sp, onChoix = { note = it })
            Text("$note / 5", style = MaterialTheme.typography.headlineMedium)
            Button(onClick = { onValider(note) }) { Text("Valider") }
        }
    }
}
```

Les étoiles cliquables viennent du composant `Etoiles(note, onChoix)` de `ui/Composants.kt`.

**Pourquoi pas une simple variable globale ?**

Parce que la boîte `savedStateHandle` est **sauvegardée par Android** : la note survit à une rotation de l'écran ou à la mise en arrière-plan de l'appli. Une variable globale, elle, serait perdue.

**Quand utiliser un ViewModel à la place ?**

Pour un petit résultat ponctuel (une note, un choix), le `savedStateHandle` est parfait. Pour des données partagées par plusieurs écrans (un panier, un utilisateur connecté), préférez un **ViewModel** commun (voir chapitre 8).

CHAPITRE 8

# Les bonnes pratiques

## 1. Ne donnez jamais le NavController à vos écrans

**À éviter**

```
@Composable
fun EcranListe(navController:
NavController) {
    Button(onClick = {

navController.navigate(Detail(1))
    }) { Text("Ouvrir") }
}
```

**À préférer**

```
@Composable
fun EcranListe(onOuvrir: (Int) ->
Unit) {
    Button(onClick = { onOuvrir(1) })
{
        Text("Ouvrir")
    }
}
```

Avec des lambdas, l'écran est **réutilisable** (même écran dans deux parcours différents), **testable**, et visible dans l'**aperçu** `@Preview` d'Android Studio sans faux NavController. Toute la navigation reste au même endroit : le NavHost.

## 2. Un seul NavHost par « niveau »

En général, un seul `NavHost` dans l'Activity suffit. Créez le NavController **une fois**, tout en haut, avec `rememberNavController()` — jamais dans un écran.

## 3. Regrouper des écrans : les graphes imbriqués

Quand l'application grandit, on peut regrouper des écrans liés (connexion, inscription, mot de passe oublié) dans un sous-graphe :

```
@Serializable object Authentification   // le nom du groupe

NavHost(navController, startDestination = Authentification) {
    navigation<Authentification>(startDestination = Connexion) {
        composable<Connexion> { /* ... */ }
        composable<Inscription> { /* ... */ }
    }
    composable<Accueil> { /* ... */ }
}

// Après la connexion, on retire tout le groupe d'un coup :
navController.navigate(Accueil) { popUpTo<Authentification> { inclusive = true } }
```

## 4. Pour aller plus loin : lire l'argument dans un ViewModel

Dans une vraie application, l'écran de détail a souvent un ViewModel. Il peut lire l'argument lui-même grâce à son `SavedStateHandle` (dépendance `lifecycle-viewmodel-compose`) :

```
class DetailViewModel(savedStateHandle: SavedStateHandle) : ViewModel() {
    private val detail: Detail = savedStateHandle.toRoute()
    val recette = recetteParId(detail.id)
}

composable<Detail> {
    val vm: DetailViewModel = viewModel()
    EcranDetail(recette = vm.recette, /* ... */)
}
```

**Liens profonds (deep links)**

Navigation Compose permet aussi d'ouvrir directement un écran depuis un lien web ou une notification ( `deepLinks = listOf(navDeepLink<Detail>(basePath = "https://...")) `). C'est une étape suivante, une fois les bases maîtrisées.

CHAPITRE 9

# Les erreurs fréquentes du débutant

**1. « Serializer for class 'Detail' is not found »**

Il manque le plugin de sérialisation ou l'annotation.
**Solution :** ajoutez `@Serializable` au-dessus de chaque destination, et le plugin `kotlin.serialization` dans Gradle (chapitre 2).

**2. « Destination ... cannot be found in navigation graph »**

Vous naviguez vers une destination qui n'a pas de `composable<...>` dans le NavHost.
**Solution :** ajoutez-la dans le bloc `NavHost { }`.

**3. Le bouton Retour ramène à l'écran de connexion**

**Solution :** `popUpTo<Connexion> { inclusive = true }` lors du `navigate` (chapitre 5).

**4. Les onglets s'empilent et Retour parcourt tous les onglets visités**

**Solution :** utilisez les trois options `popUpTo(...) { saveState = true }`, `launchSingleTop` et `restoreState` (chapitre 6).

**5. Un écran s'ouvre deux fois après un double clic**

**Solution :** `launchSingleTop = true`, ou désactivez le bouton pendant la navigation.

**6. Le contenu passe sous la barre du bas**

**Solution :** transmettez le `padding` du Scaffold au NavHost : `NavHost(..., modifier = Modifier.padding(padding))`.

**7. Le NavController est recréé et l'appli revient au début**

Vous avez écrit `NavHostController(context)` ou créé le contrôleur dans un écran.
**Solution :** `val navController = rememberNavController()`, une seule fois, tout en haut.

**8. « Unresolved reference: toRoute / composable »**

Il manque un import.
**Solution :** `import androidx.navigation.toRoute` et `import androidx.navigation.compose.composable`. Dans Android Studio, **Alt + Entrée** sur le mot en rouge ajoute l'import.

CHAPITRE 10

# Exercices corrigés

Les solutions sont déjà intégrées au projet **MesRecettes** (commentaires `// exercice N`) : essayez d'abord sans regarder !

EXERCICE 1 · FACILE — DANS EXEMPLE1

Ajoutez un écran **Paramètres** (avec un interrupteur « Notifications »), accessible par un bouton depuis l'écran Profil.

### Correction

```
// Routes.kt
@Serializable
object Parametres

// Dans le NavHost
composable<Profil> {
    EcranProfil(
        onRetour = { navController.popBackStack() },
        onParametres = { navController.navigate(Parametres) }
    )
}
composable<Parametres> {
    EcranParametres(onRetour = { navController.popBackStack() })
}
```

Pile après les deux clics : [Accueil, Profil, Parametres]. Deux appuis sur Retour ramènent à l'accueil.

EXERCICE 2 · MOYEN — DANS EXEMPLE2

Sur le détail d'une recette, ajoutez un bouton « Voir l'auteur » qui ouvre un écran **Auteur** affichant le nom de l'auteur et son nombre de recettes.

### Correction

```
@Serializable
data class Auteur(val nom: String)

// Dans composable<Detail> :
onVoirAuteur = { navController.navigate(Auteur(nom = recette.auteur)) }

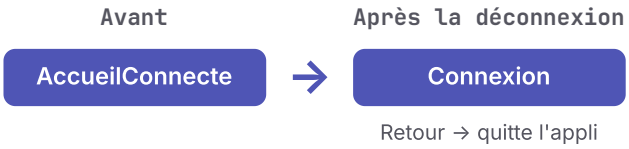
composable<Auteur> { backStackEntry ->
    val auteur: Auteur = backStackEntry.toRoute()
    EcranAuteur(nom = auteur.nom, onRetour = { navController.popBackStack() })
}
```

EXERCICE 3 · DÉFI — DANS EXEMPLE3

Ajoutez un bouton « Se déconnecter » sur l'accueil connecté. Il ramène à l'écran de connexion, et Retour doit alors **quitter** l'application.

### Correction

```
onDeconnexion = {                                    // exercice 3
    navController.navigate(Connexion) {
        // Vide TOUTE la pile, puis affiche Connexion
        popUpTo(navController.graph.id) { inclusive = true }
    }
}
```




Retour → quitte l'appli

CHAPITRE 11

# Aide-mémoire

## Déclarer des destinations

```
@Serializable object Accueil                       // sans argument
@Serializable data class Detail(val id: Int)       // avec argument
```

## Le NavHost

```
val navController = rememberNavController()
NavHost(navController, startDestination = Accueil) {
    composable<Accueil> { EcranAccueil(onOuvrir = { navController.navigate(Detail(it)) }) }
    composable<Detail>  { entry -> EcranDetail(id = entry.toRoute<Detail>().id) }
}
```

## Naviguer

```
navController.navigate(Detail(42))      // avancer
navController.popBackStack()            // reculer
navController.navigateUp()              // flèche ← de la barre du haut
```

## Gérer la pile

```
navController.navigate(Accueil) {
    popUpTo<Connexion> { inclusive = true }   // retirer Connexion
    launchSingleTop = true                    // pas de doublon
}
```

## Onglets

```
navController.navigate(onglet.route) {
    popUpTo(navController.graph.findStartDestination().id) { saveState = true }
    launchSingleTop = true
    restoreState = true
}
```

## Renvoyer un résultat

```
navController.previousBackStackEntry?.savedStateHandle?.set("note", 4)   // écran qui répond
val note by entry.savedStateHandle.getStateFlow("note", 0).collectAsState() // écran qui
attend
```

## Check-list

- ☐ Plugin `kotlin.serialization` + dépendance `navigation-compose` ajoutés.
- ☐ Chaque destination est un `object` ou une `data class` avec `@Serializable`.

- ☐ Chaque destination a son `composable<...>` dans le NavHost.
- ☐ Le NavController est créé une seule fois avec `rememberNavController()`.
- ☐ Les écrans reçoivent des lambdas, pas le NavController.
- ☐ On transmet des identifiants, pas des objets entiers.
- ☐ Les écrans à ne pas revoir (connexion, accueil de bienvenue) sont retirés avec `popUpTo`.

CHAPITRE 12

# Le projet Android Studio

## Ouvrir et lancer le projet

1. Décompressez l'archive du cours.
2. Dans Android Studio : **File › Open…** puis choisissez le dossier `projet/MesRecettes`.
3. Attendez la fin de la synchronisation Gradle (barre de progression en bas).
4. Choisissez un émulateur ou branchez un téléphone, puis cliquez sur ▶ **Run 'app'**.

Au lancement, un **menu** propose les cinq exemples du cours. Chaque exemple est indépendant, dans son propre dossier.

| Dossier | Chapitre | Ce que vous y trouvez |
|---|---|---|
| `data/` | — | Les recettes ( `Recette` , `recettes` , `recetteParId` ). |
| `ui/` | — | Composants communs : `BarreDuHaut` , `CarteRecette` , `ListeDeRecettes` , `Etoiles` . |
| `exemple1/` | 3 | Accueil → Profil (+ exercice 1 : Paramètres). |
| `exemple2/` | 4 | Liste → Détail avec argument (+ exercice 2 : Auteur). |
| `exemple3/` | 5 | Connexion avec `popUpTo` (+ exercice 3 : déconnexion). |
| `exemple4/` | 6 | Barre d'onglets en bas. |
| `exemple5/` | 7 | Noter une recette : renvoyer un résultat. |

## Configuration

- Android Gradle Plugin 8.7.3, Gradle 8.11.1, Kotlin 2.0.21
- Navigation Compose 2.8.5, Compose BOM 2024.12.01
- minSdk 24 (Android 7.0), targetSdk 35

> **Expérimentez !**
> Le meilleur moyen d'apprendre : modifiez le code. Retirez `inclusive = true` dans l'exemple 3, ou `restoreState = true` dans l'exemple 4, relancez, et observez la différence avec le bouton Retour.

> **Pour aller plus loin**
> Documentation officielle Android (developer.android.com) : « Navigation avec Compose », « Sûreté du typage dans Navigation » et « Barre de navigation ».

## Bravo, vous savez maintenant naviguer dans une application Compose ! 🎉